CEG5206 · Data Layout and Communication Cost of Distributed Matrix Multiplication

Project Proposal (Condensed)

Data Layout and Communication Cost of Distributed Matrix Multiplication

Team (4 members): Yang Tingjie, Yang Hangwei, Meng Xiangrun, Jin Ceyang

Duration 6 weeks Platform: logical multi-node execution on a single CPU with BSP cost analysis

1 Project Overview and Research Question

This project implements 1D ring, 2D Cannon and data-replicated 2.5D matrix multiplication, and studies how the layout changes the communication volume, the number of synchronization rounds and the per-node memory, comparing them under unified communication parameters and a memory budget. The three algorithms are verified with real matrix data and the cost is recorded with auditable supersteps; the project delivers a reproducible layout-selection chart, a cost breakdown and an analysis of model applicability.

The core question is: for a given matrix size, number of logical nodes and memory budget, is the communication saved by replication enough to offset the costs of replication, reduction and layout conversion? Does the optimal layout change as the communication cost rises, and where does the change occur? It is not presupposed that all three have an optimal region.

Dense matrix multiplication is the core computation of neural-network linear layers; this project studies its distributed execution, providing a basis for understanding the compute–communication trade-off in tensor parallelism. Square matrix multiplication is a controlled workload; it does not represent a complete training/inference system, nor is it equivalent to gradient all-reduce in data parallelism. (Course-concept alignment: reference [10].)

2 Computation, Platform Contract and BSP Cost Model

- The operation is $C = AB$, with A, B, C being $n \times n$ dense matrices; inputs and outputs use FP32, a multiply-add counts as 2 FLOPs, and the reference implementation uses FP64 accumulation.
- Row-major order, element numbering and logical node numbering are fixed; each node accesses only its own storage, and cross-node data is passed through an explicit message-copy interface.
- C/C++20 with one threading mechanism; the three layouts share a single self-written blocked GEMM kernel, and compiler optimization, SIMD settings and local block size are frozen, to avoid mistaking kernel differences for communication gains.
- P is the number of logical nodes and H the number of physical threads, always reported separately; when $P > H$, batched execution is allowed, for correctness and counting only, and its wall-clock time does not represent the parallel time of P machines.
- The scope includes the three layouts, the message plan, exact counting, parameter sweeps and limited single-machine validation; complex network queueing, fault recovery, full training and multiple collective-communication algorithms are not part of the core.

BSP cost model. Each superstep is executed in three phases: local computation, message exchange and synchronization end; let $F(r, s)$ be the FLOPs of node r in superstep s and $h(r, s)$ the larger of its sent and received payload word counts, then

$$T_{\mathrm{BSP}} = \sum_s \left[w \max_r F(r,s) + g \max_r h(r,s) + l \right], \qquad W_{\mathrm{crit}} = \sum_s \max_r F, \quad C_{\mathrm{crit}} = \sum_s \max_r h,$$

hence $T_{\mathrm{BSP}} = wW_{\mathrm{crit}} + gC_{\mathrm{crit}} + lS$. Here $w/g/l$ are the per-FLOP, per-word and per-synchronization costs, calibrated by microbenchmarks; $g = \gamma w_0$, $l = \lambda w_0$, and the normalized cost T_{BSP}/w_0 is reported. The model assumes equal-speed nodes and an idealized fully connected, symmetric full-duplex network, and does not simulate topology congestion, finite queues or compute–communication overlap; g is charged only per amount of data and l only per superstep. The global F_{total} and C_{total} are also reported (they cannot replace the critical-phase counts). The communication lower bound $\Omega\left(n^3/(P\sqrt{M})\right)$ [6, 7] is cited directly as an order-of-magnitude reference; no constant is assigned and no percentage of the lower bound is computed.

3 Three Layouts and Fair Comparison

1D ring. A, B, C are partitioned into contiguous row slices; each node keeps its A and C row slices and holds one B row slice, updates C with the corresponding column interval of A in each round, and then rotates the B row slice to the next node; P local updates and $P-1$ ring shifts in total. B is not replicated to every node, and the receive buffer is counted in the peak memory.
2D Cannon. A $q \times q$ grid, $P = q^2$; A, B are distributed in two-dimensional blocks, and after an initial skew, q local multiplications are performed with A, B blocks moved between rounds; the direction and numbering are fixed and coverage is checked block by block; both the initial skew and the inter-round moves are counted in communication, and the logical network allows direct exchange between any nodes [5].
2.5D replication. A $q \times q \times c$ grid, $P = cq^2$, $c \mid q$; the Cannon extension of Solomonik–Demmel is adopted [4]: A, B are replicated from a base layer, each layer performs a different q/c segment of the multiplication, and finally C is summed along the layer direction by a fixed binary tree; all inner-product terms of each output block must cover exactly once; $c = 1$ coincides with 2D and is not treated as a separate result.

P	2D grid	Additional 2.5D configuration	Multiplication rounds per layer q/c
16	4×4	none	n/a
64	8×8	$4 \times 4 \times 4$	1
256	16×16	$8 \times 8 \times 4$	2

The above satisfies $c \le P^{1/3}$ and the divisibility conditions; P=64, c=4 is the 3D endpoint and P=256, c=4 a non-endpoint case; larger logical-node counts are executed sequentially or in batches on a single machine, without requiring 256 physical cores.
Unified input/output and memory budget. All routes take the same 1D row slices as the common input/output; the end-to-end metric includes distribution conversion, packing, initial skew, replication, local multiplication, reduction and output conversion, and the native-layout multiplication-phase cost is reported separately. The per-node storage order is $\Theta(n^2/P)$ for 1D/2D and $\Theta(cn^2/P)$ for 2.5D (plus buffers); the native communication order is $\Theta(n^2)$ for 1D ($P > 1$), $\Theta(n^2/\sqrt{P})$ for 2D and $O(n^2/\sqrt{Pc})$ for 2.5D. The simultaneously live peak of input/output/replicas/temporary buffers is recorded; the layouts compete under the same per-node budget M_{cap}, and those exceeding it are marked infeasible.

4 Required Tasks and Correctness

Task 1 Data structures and reference implementation. Matrix sharding, global-to-local indexing, explicit ownership and a common row-shard interface; a self-written blocked FP32 kernel plus an independent FP64 oracle; first verify single-node multiplication, then verify the round-trip identity of the distribution conversion; messages and matrix objects have clear lifecycles, and node code must not read others' data through shared pointers.
Task 2 Superstep executor and auditable records. Implement `send/deliver/step_end` with a fixed tree broadcast and tree reduction; the single-machine executor schedules logical nodes

superstep by superstep and copies real payloads, recording the node, phase, source, destination, element interval, message word count and local FLOPs, with counts merged from local records; each superstep checks send–receive matching, payload length, message uniqueness and buffer bounds; an independent small-scale hand-computed trace is used for cross-checking; the core does not use sleep or busy-waiting to inject virtual network time.

Task 3 Three algorithms and layout conversion. First complete the 1D ring and 2D Cannon, then implement the layer replication, inner-product intervals and result reduction of 2.5D; all routes output C in the common distribution; the actually executed block-multiplication triples (i, k, j) of 2.5D are recorded to verify no omission and no duplication, and it is checked that $c = 1$ agrees with 2D in output and schedule.

Task 4 Model analysis and memory statistics. Derive the per-phase computation, communication and synchronization counts of the chosen schedule and compare them item by item with the program records; measure the peak live payload per node, distinguishing the logical-node peak, the whole-program memory and the measured DRAM traffic; implement the parameter sweep, budget filtering, cost breakdown and layout-selection chart.

Task 5 Limited measurement and interpretation. When $P \leq$ the number of available physical cores, execute a small number of 1D/2D configurations with independent worker threads, measuring the full process and the computation, copy and synchronization times; calibrate local parameters with independent microbenchmarks and perform prediction checks on held-out matrix sizes; $P > H$ runs serve only for correctness/counting; the core evidence for 2.5D is correctness, counts and model comparison, with no multi-machine speedup claimed.

Correctness test set. Inputs include zero matrices, identity matrices, all-ones, fixed-seed bounded random values and positive–negative cancellation, covering the $P = 1$ degenerate path, valid 2.5D configurations and rejection of invalid configurations; small scales are checked element by element against FP64 with the error rule $|x - x_{\text{ref}}| \leq \text{atol} + \text{rtol}|x_{\text{ref}}|$ (initially atol $= 10^{-4}$, rtol $= 10^{-3}$), reporting the maximum absolute error and the Frobenius relative error; non-divisible sizes are zero-padded to a multiple of P and the output cropped, the main performance grid is not padded, and $n = 17, 65, 257$ plus block tails are tested, rejecting invalid P, c and grids.

5 Bounded Experimental Design

Dimension	**Frozen value**
Square size n	256, 512, 1024
Logical nodes P	16, 64, 256
Layout candidates	P=16: 1D, 2D; P=64/256: 1D, 2D, 2.5D (c=4)
Communication/computation cost $\gamma = g/w_0$	1, 10, 100, 1000 FLOPs/word
Synchronization/computation cost $\lambda = l/w_0$	10^2, 10^4 FLOPs/superstep
Per-node memory budget	$M_{\text{cap}} = \mu n^2/P$ words, $\mu = 8, 32$

In total $3 \times (2 + 3 + 3) \times 4 \times 2 = 192$ cost scenarios; the two budget levels only filter the same results, the layout-selection panel has $3 \times 3 \times 4 \times 2 \times 2 = 144$ cells, and infeasible cases are explicitly marked. The actual matrix execution covers only the $3 \times (2+3+3) = 24$ main configurations with traces saved, and changing γ or λ simply recomputes the cost; the cost sweep is deterministic and recomputing 30 times is not treated as a statistical measurement.

Single-machine measurement subset. Choose $P = 1$ and 4; measure the parallel performance of $P = 4$ only with at least 4 physical cores, otherwise keep serial and local microbenchmarks and state the limitation; sizes are 256/512/1024; $P = 1$ is the shared-kernel baseline; the thread count, binding and background load are fixed and recorded. Before timing, warm up 5 times and collect 15 repetitions, reporting the median and interquartile range [9]; for the single-machine methodology see [8]. Input generation, allocation, file I/O and validation are excluded; the necessary packing,

layout conversion, message copy, computation and reduction on the route are included; storage is pre-allocated but its memory peak is counted; per-phase times are also reported, and the full-process time prevails under disturbance.

Required plots and metrics. (1) End-to-end layout-selection chart (faceted by n, λ and budget, γ on the horizontal axis and P on the vertical, ties allowed); (2) stacked computation/communication/synchronization chart and a breakdown of the common conversion and algorithm body; (3) per-node peak-memory comparison marking the infeasible region (at least P=256, c=4); (4) comparison of the native and common input/output metrics; (5) a few single-machine measurements and prediction residuals. GFLOP/s $= 2n^3/(10^9 t)$ is reported only for real timing; model speed ratios are labelled as predictions; measured parallel efficiency is computed with the real thread count H, not the oversubscribed logical P.

6 Model Validation and Conclusion Boundary

The three layouts process the same input and produce the same mathematical output, checked by the agreed error rule; the message and block-coverage checks are independent evidence. For small configurations the per-step FLOPs, send/receive volumes and synchronization counts are hand-computed and checked against the program trace; comparing analytic with actual counts is implementation validation and does not claim to validate a real network model. Local-computation, shared-memory-copy and barrier parameters are obtained from independent microbenchmarks, and predictions are checked on full sizes that did not take part in the calibration; the explanation is limited to this shared-memory implementation, `memcpy` bandwidth is not treated as network bandwidth, and significant errors are reported as residuals and causes rather than hidden by per-configuration refitting. Conclusions are stated as "under the listed BSP assumptions, budgets and parameters, a certain layout has a lower cost"; cluster conclusions would still need topology, contention and overlap measurements; cost breakdowns, predictions and real timings use different legends and column names.

7 Six-Week Plan, Roles and Deliverables

Week	Work and checkpoint
Week 1	Freeze the contract, valid grids and superstep semantics; complete the FP64 oracle, shared kernel and round-trip conversion tests
Week 2	Complete 1D and 2D; hand-compute small configurations and check the communication traces; preliminary memory statistics
Week 3	Complete 2.5D, tree replication and reduction; pass coverage checks and correctness validation of the 24 main configurations
Week 4	Freeze counts and implementation; complete the parameter sweep, budget filtering and single-machine validation subset
Week 5	Complete the plots, conversion-cost ablation and model-residual analysis; organise raw data and limitations
Week 6	Reproduce from a clean environment; complete the report, demonstration and cross-review

Member	Primary responsibility	Cross-review
A	Data contract, shared kernel, FP64 oracle, numerical tests	2.5D multiplication coverage and reduction
B	1D/2D, distribution conversion, hand-computed communication counts	End-to-end timing and budget fairness
C	2.5D, replication/reduction, memory lifecycle	Message matching and core model assumptions
D	Superstep executor, parameter sweep, single-machine calibration, plotting	Oracle independence and experimental reproduction

In week 1 the message and local-matrix interfaces are defined jointly, and integration takes place weekly from week 2; every member undertakes code, validation and interpretation; the algorithm scope is frozen after week 3, and extensions must not replace the incomplete three layouts.

Workload control. One shared kernel, one 2D algorithm, one replication factor, one tree-communication scheme and one phased executor; the general network simulator, finite queues, fault recovery and multiple all-reduce algorithms are dropped; after the core is complete, at most one extension is added (rectangular linear-layer workloads, or a small number of real multi-process/multi-machine trend checks), which does not change the acceptance conditions.

Minimum completion criteria. (1) The three layouts execute real data, return the common row-shard output, and pass the numerical/message/block-coverage checks; (2) auditable per-phase counts and per-node peak storage are output, the independent hand-computed cases agree, and the $c = 1$ degenerate path is correct; (3) the 192 scenarios and two-level budget filtering are completed, with the metrics and assumptions of each plot made explicit; (4) single-machine validation within the available hardware is provided, or the reason for its absence is stated, with model/count/real-timing conclusions presented separately; (5) one representative reproduction is completed from a clean build. A fixed speedup, a new lower-bound proof, or a full-cluster deployment are not acceptance conditions.

Deliverables. Source code and build (C/C++20, CMake/Makefile, compiler/flags/dependencies/commands); experimental configuration (fixed seed, list of valid grids, local block size, model parameters and memory budget); raw data (layout traces, model CSV, measurement CSV, correctness summary, hardware and thread configuration); plots and reproduction (plotting scripts; one command for a small-scale test, the counts and a representative model chart); technical report (suggested 8–10 pages, excluding references); demonstration (suggested 10 minutes). It is suggested that the CSV record n, P, H, layout, c, phase, $F_{\text{total}}, W_{\text{crit}}, C_{\text{total}}, C_{\text{crit}}, S$, `peak_words`, $\gamma, \lambda, T_{\text{model}}$, `t_measured`; rows without measurement leave `t_measured` empty rather than filling it with predictions; the layout-selection table records both native and end-to-end costs and the memory feasibility.

8 References

References

[1] J. M. D. Hill et al. *BSPlib: The BSP Programming Library*. Parallel Computing, 24(14), 1998, pp. 1947–1980. For supersteps, h-relation and performance parameters.
`https://research-portal.uu.nl/files/62571999/BSPlib.pdf`

[2] L. G. Valiant. *A Bridging Model for Parallel Computation*. Communications of the ACM, 33(8):103–111, 1990. Original definition of BSP and supersteps.
`https://doi.org/10.1145/79173.79181`

[3] D. Culler et al. *LogP: Towards a Realistic Model of Parallel Computation*. PPoPP 1993, 1–12. A pragmatic latency/bandwidth/overhead model.
`https://doi.org/10.1145/155332.155333`

[4] E. Solomonik and J. Demmel. *Communication-optimal parallel 2.5D matrix multiplication and LU factorization algorithms*. UC Berkeley TR UCB/EECS-2011-72, 2011.
`https://www2.eecs.berkeley.edu/Pubs/TechRpts/2011/EECS-2011-72.pdf`

[5] R. A. van de Geijn and J. Watts. *SUMMA: Scalable Universal Matrix Multiplication Algorithm*. Concurrency: Practice and Experience, 9(4):255–274, 1997. 2D broadcast-based layout.
`https://doi.org/10.1002/(SICI)1096-9128(199704)9:4<255::AID-CPE250>3.0.CO;2-2`

[6] G. Ballard, J. Demmel, O. Holtz and O. Schwartz. *Minimizing Communication in Numerical Linear Algebra*. SIAM J. Matrix Anal. Appl., 32(3), 2011, pp. 866–901.
`https://doi.org/10.1137/090769156`

[7] G. Ballard, D. Carson, J. Demmel, M. Hoemmen, N. Knight and O. Schwartz. *Communication Lower Bounds and Optimal Algorithms for Numerical Linear Algebra*. Acta Numerica, 23:1–155, 2014.
https://doi.org/10.1017/S0962492914000038

[8] T. Hoefler, T. Schneider and A. Lumsdaine. *LogGOPSim: Simulating Large-Scale Applications in the LogGOPS Model*. HPDC 2010, 597–604.
https://doi.org/10.1145/1851476.1851564

[9] T. Hoefler and R. Belli. *Scientific Benchmarking of Parallel Computing Systems*. SC15, 2015, 73:1–73:12.
https://doi.org/10.1145/2807591.2807644

[10] CEG5206 lecture notes L01, L02, L03, L06; Sample Project 1 and Sample Project 2. For alignment with course concepts, workload and experimental-report requirements.